triggered after a newly created active record object is saved to the database. Inside the block that is invoked when the callback is triggered, we call a method named broadcast_append_later_to.

The method broadcast_append_later_to is one of a family of methods defined in a module called Broadcastable, which turbo-rails mixes into every ActiveRecord class. Every part of that name is important and also suggests the other methods in the module. Let's break it down:

- broadcast: All the methods in the broadcastable module start with broadcast. All of these methods render something and then wrap the rendered text inside a turbo-stream tag.

- append: This is the action of the resulting Turbo Stream response. There are also methods for all the other Turbo Stream actions (replace, update, prepend, remove) and a generic broadcast_action family of methods that takes the action as an argument.

- later: This ensures the method performs the ActionCable broadcast asynchronously using ActiveJob. If you leave off later in the method name, the broadcast is performed in-band as part of the database save. Generally, it's best to avoid doing the render in-band as it might be slow and would delay whatever is waiting on the database save.

- to: This signifies that the first set of arguments to the method is the name of the ActionCable channel. If to is left off (as in broadcast_append_later), the name of the ActionCable channel is inferred from the identity of the object in question.

So we are rendering an action, wrapping it in an append Turbo Stream action, and then broadcasting it in the background to an ActionCable channel. The non-keyword arguments to the method are the parts of the name of the channel: user, and :favorites. The block is evaluated via instance_eval, so user, in this context, is self.user—the user attached to the favorite object that has just been created. The method takes a keyword argument called target, and the value of that argument is